

程序设计Ⅶ

张宏斌 副教授

复习

- 字符串定义；字符串编码；转义字符：\n, \\, \', \t,
- 字符串格式化：format()格式化；f表达式；center(), ljust(), rjust()方法
- 字符串索引与切片
- 字符串运算符：+, *, in
- 内置函数的应用：len(), eval(), sorted(), max(), min(),
- 字符串常用方法与操作：
 - 查找：find(), rfind(), index(), rindex(), count()
 - 分割：split(), rsplit()
 - 连接：join()
 - 大小写转换：lower(), upper(), capitalize(), title(), swapcase()
 - 替换：replace()
 - 首尾截除：strip(), lstrip(), rstrip()
 - 首尾判断：startswith(), endswith()
 - 字符串判断：isalnum(), isalpha(), isdigit(), isdecimal(), isnumeric(), isspace(), isupper(), islower()

第三章 列表和元组

- 1 掌握序列类型的基本概念
- 2 熟悉序列类型的通用操作
- 3 掌握列表类型的基本概念
- 4 熟悉列表类型的基本操作
- 5 掌握元组类型的基本概念
- 6 熟悉元组类型的基本操作
- 7 理解列表与元组的异同点
- 8 灵活运用列表和元组解决实际问题

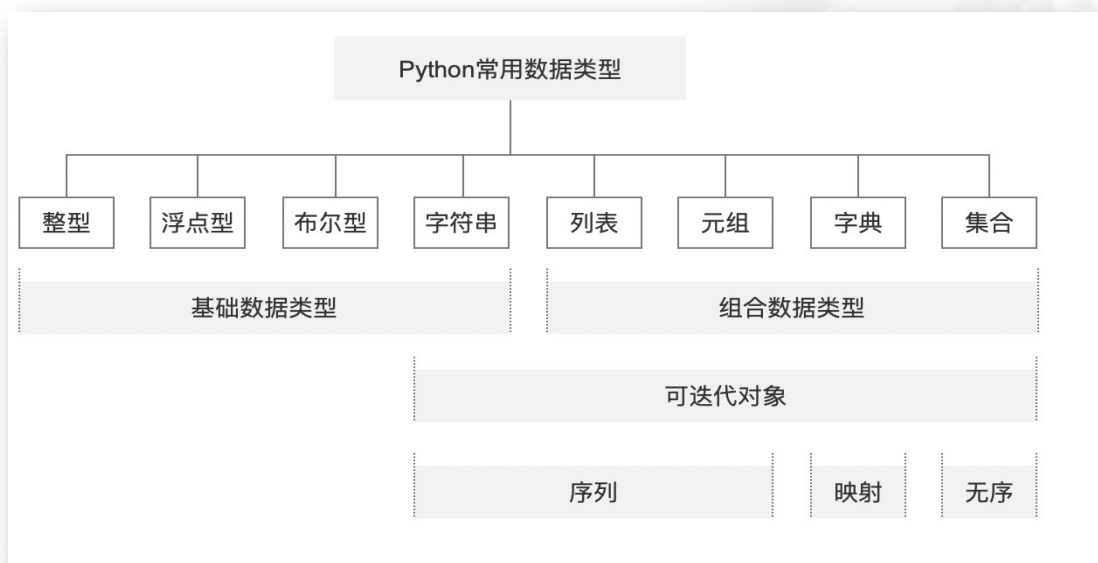


中山大學嶺南學院
LINGNAN COLLEGE SUN YAT-SEN UNIVERSITY

张宏斌 副教授 2026年春



Python常用数据类型

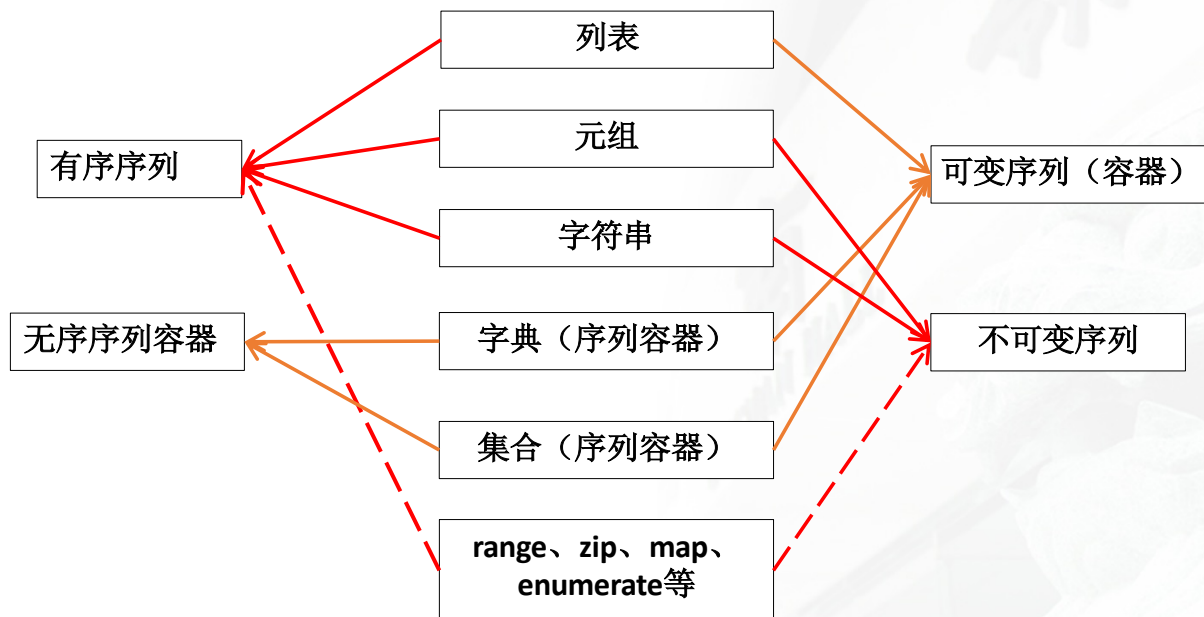


中山大學嶺南學院
LINGNAN COLLEGE SUN YAT-SEN UNIVERSITY

张宏斌 副教授 2026年春



Python常用数据类型



通用序列操作

- 索引
- 切片
- 运算符 **+** 和 *****
- 成员运算符: **in** 和 **not in**
- **count()**方法



索引

- 在字符串中已经学过
- 有序序列类型是一个元素向量，元素之间存在先后关系，所有元素都有**序号（索引）**
- **正向序号**从左向右，由 0 开始递增，直到序列长度减 1
- **反向序号**从右向左，由 -1 开始递减，直到序列元素个数的负数值



```
>>> emp_info = ['10933', '李斯', 40, [12000.00, 400.00]]
>>> emp_info[0]
'10933'
>>> emp_info[-4]
'10933'
>>> emp_info[3][-1]
400.0
```



索引（续）

- 在使用索引时，序号不能超过允许的范围，否则会触发 `IndexError` 的异常，导致程序无法继续执行

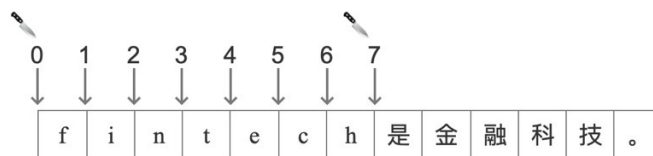
```
>>> empInfo = ['10933', '李斯', 40, [12000.00, 400.00]]
>>> empInfo[5]
Traceback (most recent call last):
  File "<pyshell#36>", line 1, in <module>
    empInfo[5]
IndexError: list index out of range
```



切片（分片）

- 当方括号中的索引变成由冒号分隔的一对序号时，Python 将返回一个包含这对序号所标识的连续内容的新对象
- 切片的内容**包含冒号左边序号的元素，没有包含冒号右边序号的元素（截止前一位）**
- 与使用索引作为下标访问列表元素的方法不同，切片操作**不会因为下标越界而抛出异常**，而是简单地在列表尾部截断或者返回一个空列表，**代码具有更强的健壮性**

分片表示形式 [开始 : 结束]



分片所得到的内容是刀在开始和结束的序号指定处“切下”后的内容。
即[0:7]将获得一个新的字符串：fintech



中山大學嶺南學院
LINGNAN COLLEGE SUN YAT-SEN UNIVERSITY

张宏斌 副教授 2026年春



切片（续）

- 假设 s 为字符串“fintech是金融科技。”，对于切片，**需要注意：**
 - **包含左边界，不包含右边界**
 - **s[:]** 获得包含所有字符的新字符串，缺省的左边界为 0，右边界为序列长度
 - **s[0: 7]** 获得新字符串“fintech”，即序号为 0 直到序号为 7 **之前**的所有字符
 - **s[8:]** 获得新字符串“金融科技。”，即从序号 8 到最后的所有字符
 - **s[:3]** 获得新字符串“fin”，即从开头直到不包括序号为 3 之间的所有字符
 - **s[:-1]** 获得新字符串“fintech是金融科技”，即从开头直到不包括最后一个字符之间所有字符



中山大學嶺南學院
LINGNAN COLLEGE SUN YAT-SEN UNIVERSITY

张宏斌 副教授 2026年春



切片——步长

- 在切片时，除了可以指定左边界和右边界的序号外，还可以增加第三个值，即**步长**
- **步长**和右边界之间也是使用冒号分隔，在不指定步长的情况下，步长默认值为 1
- **步长**将会加到每次提取元素的序号上，作为下次需要提取元素的序号
- **步长**可以是**正数**，也可以是**负数**。当步长为正数时，Python 会从序列的左边开始向右提取元素，此时分片的左边界要小于右边界；当步长为负数时，会从序列的右边开始向左提取元素，此时分片的左边界要大于右边界。否则将得到空序列



步长

```
>>> s = 'fintech是金融科技。'
>>> s[1:7:2]

'ite'

>>> s[::-1]

'。技科融金是hceetniF'

>>> s[-2:7:-2]

'技融'

>>> s[1:7:-2]

''

>>> s[-2:7:2]

''
```



运算符 + 和 *

- 对于两个类型相同的序列，使用**加号+**进行**连接**，得到一个该类型的新序列

```
>>> stock_name = ['600000', '浦发银行']
>>> stock_price = [11.52, 11.54, 11.61, 11.40]
>>> stock_info = stock_name + stock_price
>>> print(stock_info)

['600000', '浦发银行', 11.52, 11.54, 11.61, 11.4]

>>> print(stock_name)
>>> print(stock_price)

['600000', '浦发银行']
[11.52, 11.54, 11.61, 11.4]
```

- 序列类型的值和一个**整数n相乘**，得到将该序列重复 n 次新的对象

```
>>> print(['600000', '浦发银行'] * 3)
>>> print(['600036', '招商银行'] * 2)

['600000', '浦发银行', '600000', '浦发银行', '600000', '浦发银行']
['600036', '招商银行', '600036', '招商银行']
```



成员运算符：in 和 not in 判断

- 通过成员运算符 **in** 可以判断某个元素是否在序列中，如果元素在序列中，in 表达式返回 **True**，否则返回 **False**。运算符 **not in** 则正好相反

```
>>> email_subject = '<广告> 休息一下，看看有没有感兴趣的产品'
>>> '<广告>' in email_subject

True

>>> '<广告>' not in email_subject

False
```



方法count()

- 当需要计算某个元素在序列中出现的次数时，使用序列的 count 方法可以很轻松地完成任务：

```
>>> stock_down_list = ['600000', '600036', '000001', '600036', '600000', '300142', '600036']
>>> stock_down_list.count('600036')
3
```



中山大學嶺南學院
LINGNAN COLLEGE SUN YAT-SEN UNIVERSITY

张宏斌 副教授 2026年春



列表——最灵活的序列

- 在形式上，**列表**的所有元素放在**一对方括号[]**中，相邻元素之间使用逗号分隔
- 在Python中，同一个列表中**元素的数据类型可以各不相同**，可以同时包含整数、实数、字符串等基本类型的元素，也可以包含列表、元组、字典、集合、函数以及其他任意对象
- 如果只有一对方括号而没有任何元素则表示空列表

```
[10, 20, 30, 40]
['crunchy frog', 'ram bladder', 'lark vomit']
['spam', 2.0, 5, [10, 20]]
[['file1', 200,7], ['file2', 260,9]]
[{3}, {5:6}, (1, 2, 3)]
[]
```



中山大學嶺南學院
LINGNAN COLLEGE SUN YAT-SEN UNIVERSITY

张宏斌 副教授 2026年春



列表的定义

- 利用list()函数创建列表——list()函数将元组、range对象、字符串、字典、集合或其他**有限长度的可迭代对象**转换为列表：

```
>>> list('招商银行')  
['招', '商', '银', '行']
```

- list()函数的参数可以是字符串、元组、字典或者集合等**可迭代对象**，但不能是整数、浮点数或布尔值：

```
>>> list(10.8)  
Traceback (most recent call last):  
  File "<pyshell#63>", line 1, in <module>  
    list(10.8)  
TypeError: 'float' object is not iterable
```



列表的定义（续）

- 使用**方括号 []** 创建列表时，需要在列表中列出所有元素：

```
>>> ['招商银行']  
['招商银行']  
>>> ['招', '商', '银', '行']  
['招', '商', '银', '行']
```

- 可以使用 list() 函数和方括号 [] 创建**空列表**：

```
>>> list()  
[]  
>>> []  
[]
```



列表可以改变

```
x = [1, 2, 3]
x[-1:] = [4, 5, 6]      # x=[1, 2, 4, 5, 6]
x[1:-1] = []            # x=[1, 6]
x[1:1] = [2, 3, 4, 5]   # x=[1, 2, 3, 4, 5, 6], 很奇特吧!
```



列表及其元素的删除

- 当一个列表不再使用时，可以使用 **del 命令** 将其删除。既可以删除一个元素，也可以删除整个列表

```
>>> x = [1, 2, 3]
>>> del x[1]      # 删除列表的第2个元素
>>> x             # [1, 3]
>>> del x         # 删除列表对象
>>> x             # 对象删除后无法再访问，抛出异常
NameError: name 'x' is not defined
```



列表的常用方法

方法	说明
append(x)	将x追加至列表尾部
extend(L)	将 可迭代 对象L中 所有元素 追加至列表尾部
insert(index, x)	在列表 index 位置处 插入x ，该位置后面的所有元素后移并且在列表中的索引加1，如果index为正数且大于列表长度则在列表尾部追加x，如果index为负数且小于列表长度的相反数则在列表头部插入元素x
remove(x)	在列表中 删除第一个值为x的元素 ，该元素之后所有元素前移并且索引减1，如果列表中不存在x则抛出异常
pop([index])	删除并返回列表中下标为index的元素 ，如果不指定index则 默认为-1 ，弹出最后一个元素；如果弹出中间位置的元素则后面的元素索引减1；如果index不是[-L, L]区间上的整数则抛出异常
clear()	清空列表 ，删除列表中所有元素，保留列表对象
index(x)	返回列表中第一个值为x的元素的索引，若不存在值为x的元素则抛出异常
count(x)	返回x在列表中的出现次数
reverse()	对列表所有元素进行 原地 逆序， 首尾交换
sort(key=None, reverse=False)	对列表中的元素进行 原地 排序， key 用来指定 排序规则 ， reverse 为 False 表示 升序 ， True 表示 降序



方法append()、insert()、extend()

- **append()** 用于向列表尾部追加一个元素
- **insert()** 用于向列表任意指定位置插入一个元素，有两个参数，第一个表示位置，第二个是插入的对象
- **extend()** 用于将另一个**可迭代对象中的所有元素追加至当前列表的尾部**
 - 这3个方法都**没有返回值**，或者说返回空值None

```
>>> stock_info = ['600000', '浦发银行']
>>> stock_price = [11.52, 11.54, 11.61, 11.4]
>>> stock_info.extend(stock_price)
>>> stock_info
['600000', '浦发银行', 11.52, 11.54, 11.61, 11.4]
```

```
>>> stock_info = ['600000', '浦发银行']
>>> stock_price = [11.52, 11.54, 11.61, 11.4]
>>> stock_info.append(stock_price)
>>> stock_info
['600000', '浦发银行', [11.52, 11.54, 11.61, 11.4]]
```

```
>>> x = list(range(5))
>>> x
[0, 1, 2, 3, 4]
>>> x.insert(2, [12, 35])
>>> x
[0, 1, [12, 35], 2, 3, 4]
```



方法pop()、remove()

- **pop()**用于删除并**返回**指定位置（默认是最后一个）上的元素
- **remove()**用于删除列表中**第一个值与指定值相等的元素**，**没有返回值**
- 另外，还可以使用**del命令**删除列表中指定位置的元素

```
>>> x = [1, 2, 3, 4, 5, 6, 7]
>>> x.pop()          # 删除并返回尾部元素
7
>>> x.pop(0)         # 删除并返回指定位置的元素
1
>>> x = [1, 2, 1, 1, 2]
>>> x.remove(2)       # 删除首个值为2的元素
>>> del x[3]          # 删除指定位置上的元素
>>> x
[1, 1, 1]
```

方法index()

- **index()**用于返回指定元素在列表中**首次**出现的位置，如果该元素不在列表中则抛出异常
 - 列表没有**find()**方法，也没有**rindex()**方法

```
>>> x = [1, 2, 2, 3, 3, 3, 4, 4, 4, 4]
>>> x.index(2)        # 元素2在列表x中首次出现的索引
1
>>> x.index(5)        # 列表x中没有5，抛出异常
ValueError: 5 is not in list
```

方法sort()、reverse()

- 列表对象的**sort()**方法用于按照指定的规则对所有元素进行排序
- **reverse()**方法用于将列表所有元素逆序或翻转

```
>>> x = list(range(11))          # 包含11个整数的列表
>>> import random
>>> random.shuffle(x)           # 把列表x中的元素随机乱序
>>> x
[6, 0, 1, 7, 4, 3, 2, 8, 5, 10, 9]
>>> x.sort()                     # 按默认规则排序
>>> x
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
>>> x.reverse()                 # 把所有元素翻转或逆序
>>> x
[10, 9, 8, 7, 6, 5, 4, 3, 2, 1, 0]
```



内置函数sorted()

- 对列表排序也可以使用内置函数**sorted()**，它会返回一个排好序点新列表，**原列表没有改变**
- 如果要按照倒序排列，可以用参数**reverse**，还可以指定键**key**

```
x = sorted([-4, 1, -2, 3], key=abs, reverse=True)
```

```
# x为[-4, 3, -2, 1]
```



方法copy()

- **copy()**方法，生成一个新的列表，复制调用者的所有元素

【例】使用列表 stock_info 来存储股票基本信息，列表 stock_day1、stock_day2 等用于存储股票的基本信息和每天的交易信息（开盘价、收盘价、最高价、最低价）

<pre>1 stock_info = ['600000', '浦发银行'] 2 # 将基本信息赋值给 stock_day1 3 stock_day1 = stock_info 4 # 在 stock_day1 添加第 1 天的交易信息 5 stock_day1.extend([11.52, 11.54, 11.61, 11.40]) 6 # 将基本信息赋值给 stock_day2 7 stock_day2 = stock_info 8 # 在 stock_day2 添加第 2 天的交易信息 9 stock_day2.extend([11.54, 11.98, 12.11, 11.20]) 10 print(stock_day1) 11 print(stock_day2)</pre>	<pre>stock_info = ['600000', '浦发银行'] # 利用 copy 复制后再赋值 1 stock_day1 = stock_info.copy() # 第 1 天的交易信息 stock_day1.extend([11.52, 11.54, 11.61, 11.40]) # 利用 copy 复制后再赋值 2 stock_day2 = stock_info.copy() # 第 2 天的交易信息 stock_day2.extend([11.54, 11.98, 12.11, 11.20]) print(stock_day1) print(stock_day2)</pre>
<pre>['600000', '浦发银行', 11.52, 11.54, 11.61, 11.4, 11.54, 11.98, 12.11, 11.2] ['600000', '浦发银行', 11.52, 11.54, 11.61, 11.4, 11.54, 11.98, 12.11, 11.2]</pre>	<pre>['600000', '浦发银行', 11.52, 11.54, 11.61, 11.4] ['600000', '浦发银行', 11.54, 11.98, 12.11, 11.2]</pre>



内置函数对列表的操作

- **max()**、**min()**函数用于返回列表中所有元素的最大值和最小值
- **sorted()**函数返回排序后的列表（原列表不变）
- **sum()**函数用于返回列表中所有元素之和
- **len()**函数用于返回列表中元素个数
- **zip()**函数用于将多个列表中元素重新组合为元组并返回包含这些元组的zip对象
- **enumerate()**函数返回包含若干下标和值的迭代对象
- **map()**函数把函数映射到列表上的每个元素
- **filter()**函数根据指定函数的返回值对列表元素进行过滤
- **all()**函数用来测试列表中是否所有元素都等价于True
all(x > 3 for x in [2, 3, 5, 8, 9])
- **any()**用来测试列表中是否有等价于True的元素



内置函数range()

- 内置函数**range()**返回range对象，range对象为**整数序列**，其中包含左闭右开区间**[start, end)**内以step为步长的整数，用法为：

- **range**([start,] end [, step])

- **range()**函数**参数必须为整数**，不允许使用浮点数和其他类型

```
for x in range(2, -1, -1):  
    print(x)          # 2, 1, 0  
list(range(0, 11, 2))  # [0, 2, 4, 6, 8, 10]
```

列表推导式（列表解析）

- **列表推导式**使用非常简洁的方式来快速生成满足特定需求的列表，代码具有非常强的可读性
- **务必熟练掌握列表推导的用法！！！！**
- 列表推导式语法形式为：

```
[expression for expr1 in sequence1 if condition1  
    for expr2 in sequence2 if condition2  
    for expr3 in sequence3 if condition3  
    ...  
    for exprN in sequenceN if conditionN]
```

列表推导式（续）

- 列表推导式在逻辑上等价于一个循环语句，只是形式上更加简洁。例如：

```
>>> aList = [x * x for x in range(10)]
```

相当于：

```
>>> aList = []
```

```
>>> for x in range(10):  
    aList.append(x * x)
```



列表推导式使用案例

```
>>> freshfruit = ['banana', 'loganberry', 'passion fruit']
```

```
>>> aList = [w.strip() for w in freshfruit]
```

- 对英文分词的启示？

- 在列表推导式中使用if子句对列表中的元素进行筛选，只在结果列表中保留符合条件的元素

```
>>> aList = [-1, -4, 6, 7.5, -2.3, 9, -11]
```

```
>>> [i for i in aList if i>0] # 所有大于0的数字
```

```
[6, 7.5, 9]
```

- 在列表推导式中同时遍历多个列表或可迭代对象

```
>>> [(x, y) for x in [1, 2, 3] for y in [3, 1, 4] if x != y]
```

```
[(1, 3), (1, 4), (2, 3), (2, 1), (2, 4), (3, 1), (3, 4)]
```

```
>>> [(x, y) for x in [1, 2, 3] if x==1 for y in [3, 1, 4] if y!=x]
```

```
[(1, 3), (1, 4)]
```



遍历列表

- 一般用**for**循环**遍历**列表，从列表第一个元素开始，每次经过一个元素，直至遍历所有元素
- 如求数字列表中所有元素的和（不用sum()函数）和乘积：

```
numlist = [25, -8, 69, 76, -15]
total_sum = 0
total_prod = 1
for num in numlist:
    total_sum += num
    total_prod *= num
print(f'列表{numlist}所有元素的和为{total_sum}, 乘积为{total_prod:,.2f}')
```

求和，一般初始值设为0
求乘积的初始值则设为1

字符串作业主要问题

元组

- 从形式上，元组的所有元素放在一对**圆括号**中，元素之间使用**逗号**分隔，元组中只有一个元素时必须在最后增加一个**逗号**
- 元组一旦定义，不可改变

(1, 2, 3)

(5,)



中山大學嶺南學院
LINGNAN COLLEGE SUN YAT-SEN UNIVERSITY

张宏斌 副教授 2026年春



元组的定义

- 使用 **tuple ()** 函数可以将参数中的可迭代对象转换成元组：

```
>>> tuple('浦发银行')  
('浦', '发', '银', '行')  
  
>>> tuple(['浦发银行', '招商银行', '保利地产', '平安银行'])  
('浦发银行', '招商银行', '保利地产', '平安银行')
```

- 使用**圆括号 ()** 创建元组时，需要在元组中列出所有元素，直接使用圆括号 () 将得到一个空的元组：

```
>>> ('浦发银行', '招商银行', '工商银行', '农业银行')  
('浦发银行', '招商银行', '工商银行', '农业银行')  
  
>>> ()  
()
```



中山大學嶺南學院
LINGNAN COLLEGE SUN YAT-SEN UNIVERSITY

张宏斌 副教授 2026年春



元组的定义（续）

- 在不引起二义性的情况下，可以省略圆括号

```
>>> stock_info = '600000', '浦发银行', 11.52, 11.54, 11.61, 11.4
>>> stock_info
('600000', '浦发银行', 11.52, 11.54, 11.61, 11.4)
>>> type(stock_info)
<class 'tuple'>
```



元组是不可变的

- 不可变性**，是元组和列表最大的区别。如果试图修改元组的某个元素，将会触发 `TypeError` 的异常

```
>>> bank_name = tuple(['浦发银行', '招商银行', '保利地产', '平安银行'])
>>> bank_name[1] = '农业银行'
```

```
Traceback (most recent call last):
  File "<pyshell#6>", line 1, in <module>
    bank_name[1] = '农业银行'
TypeError: 'tuple' object does not support item assignment
```

- 元组的不可变性仅指的是不可以改变元组的顶层元素。如果元组中的**元素本身是可变的**（例如，元组中的某个元素是列表），`Python` 允许修改其内容

```
>>> rate_info = ([ '活期存款', 0.0030 ], [ '整存整取一年', 0.0195 ],
                [ '整存整取二年', 0.0240 ], [ '整存整取三年', 0.0280 ])
>>> rate_info[1][1] = 0.0200
>>> rate_info
([ '活期存款', 0.003 ], [ '整存整取一年', 0.02 ], [ '整存整取二年', 0.024 ], [ '整存整取三年', 0.028 ])
```



多重赋值

- 在 Python 中，允许一次给多个变量赋值，称为**多重赋值（序列解包）**。可以使用元组和列表实现多重赋值：

```
>>> stock_code, stock_name = ('600000', '浦发银行')
>>> stock_code
'600000'
>>> stock_name
'浦发银行'
>>> p_open, p_close, p_high, p_low = [11.52, 11.54, 11.61, 11.4]
>>> p_open
11.52
>>> p_high
11.61
```



为什么需要元组

- 元组可以作为**字典的键或集合的元素**，但列表不行。因为列表是可变的
- 函数中利用元组传递参数**或作为返回值，可以有效地避免被意外修改
- 相对于列表，使用元组存储数据时，**访问的速度更快**



总结

- Python序列：有序 vs 无序；可变 vs 不可变
- 序列的操作：索引与切片（列表可利用切片修改、添加、删除元素）；+，*，in，not in，count()
- 列表
 - 创建与删除
 - 常用方法
 - 添加：append(), insert(), extend()
 - 删除：pop(), remove(), del命令
 - 寻找：index(), count()
 - 排序：sort(), reverse()
 - 内置函数：max(), min(), sum(), len(),
 - 列表推导式
 - 遍历列表



中山大學嶺南學院
LINGNAN COLLEGE SUN YAT-SEN UNIVERSITY

张宏斌 副教授 2026年春



总结

- 元组
 - 元组定义
 - 多重赋值（序列解包）
 - 元组的用途



中山大學嶺南學院
LINGNAN COLLEGE SUN YAT-SEN UNIVERSITY

张宏斌 副教授 2026年春



作业

- 写程序，完成以下任务：
 - 创建一个列表，包含数字400~499（包含这两个，下同）
 - 创建另一个列表包含数字3000~3500
 - 合并两个列表
 - 在合并后的列表中第4位插入826（原第4个数后移）
 - 在上面列表的第12位一次插入三个数766，555，666（原来第12位数后移）
 - 计算并输出上面最新列表（指上一步得到的列表，下同）中的最小、最大数、总和
 - 输出新列表中的偶数组成的列表
 - 输出新列表中每个数的立方组成的列表，并求其和
 - 对新列表进行排序，要求输出排序好的列表，新列表并未改变
 - 按照倒序对新列表进行排序，要求改变新列表



中山大學嶺南學院
LINGNAN COLLEGE SUN YAT-SEN UNIVERSITY

张宏斌 副教授 2026年春



AACSB
ACCREDITED



ASSOCIATION
AMBA
ACCREDITED

作业要求

- 作业要求是.py文件，务必命名为：**hw02_学号.py**
- 作业务必按照模版**hw02_template.py**（**课件中写错**）的格式写，模版在课件网上下载，建议直接在模版上完成作业，**务必记得修改文件名称**
- **务必注意：作业中的变量一旦赋值，除非明确要求，否则绝对不能改变！！！！**
- 作业截止时间为下周二晚上11:50



中山大學嶺南學院
LINGNAN COLLEGE SUN YAT-SEN UNIVERSITY

张宏斌 副教授 2026年春



AACSB
ACCREDITED



ASSOCIATION
AMBA
ACCREDITED

作业（自愿作业）

- 总结列表的定义方法、索引与切片方法、常用操作方法、列表推导式等
- 可举例



中山大學嶺南學院
LINGNAN COLLEGE SUN YAT-SEN UNIVERSITY

张宏斌 副教授 2026年春



AACSB
ACCREDITED



ASSOCIATION
AMBA
ACCREDITED